# COGNIPATH AI: ADAPTIVE DEEP KNOWLEDGE TRACING & PERSONALIZED RECOMMENDATION SYSTEM

## A DEEP LEARNING PROJECT REPORT

**Submitted in partial fulfillment of the**
**requirement for the award of the**
**Degree of**
**B.TECH**
**IN**
**COMPUTER SCIENCE AND ENGINEERING**
**by**

**Chirag Pradhan (24BCE7237)**
**Darga Syed Javid (24BCA7689)**
**Shaik Mansoor (24BCA7204)**
**Boyina Arunkamal (24BCA7881)**




**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**
**VIT-AP UNIVERSITY**
**AMARAVATI - 522237**
**Academic Year: 2025–2026**

# ABSTRACT

In this project, an intelligent, adaptive educational system titled **“CogniPath AI”** is presented that continuously models, predicts, and guides student knowledge acquisition across complex computer science and data structure curricula. Conventional learning management systems rely on uniform grading metrics and aggregate percentages, triggering the **Uniform Assessment Paradox**: two students who both achieve an aggregate score of 70% are treated identically despite having completely opposite knowledge deficits (e.g., strong in linear arrays but failing in recursive trees). To overcome this fundamental limitation, CogniPath AI introduces an end-to-end **Deep Knowledge Tracing (DKT)** framework implemented with a 2-layer PyTorch Long Short-Term Memory (LSTM) recurrent neural network.

The proposed deep learning architecture ingests multi-dimensional cognitive telemetry at each interaction step—including discrete concept identifiers, binary answer correctness, continuous problem-solving response latency, submission attempt counts, and temporal inactivity intervals. Continuous behavioral features are projected through non-linear multi-layer perceptrons with SiLU activations and normalized via LayerNorm to prevent gradient saturation. Across sequence steps, the latent recurrent memory vector $\mathbf{h}_t$ dynamically updates and outputs multi-concept mastery probabilities via a sigmoid prediction head.

CogniPath AI connects these deep sequence predictions directly to a pedagogical diagnostic suite featuring: (1) an explainable Weakness & Prerequisite DAG Blocker Detector that traces conceptual root causes ('Why You Are Weak'), (2) Hermann Ebbinghaus forgetting curve modeling with memory stability tracking to trigger automated spaced-repetition revision, (3) an AI Learning DNA profiler categorizing methodical vs. intuitive problem solvers, and (4) an adaptive recommendation engine operating within Vygotsky’s Zone of Proximal Development (ZPD). Empirical validation on the CogniPath CS-DKT benchmark demonstrates **89.31% Accuracy** and a **0.9315 ROC-AUC score**, while generalizability testing on 525,534 interactions from the ASSISTments benchmark delivers **0.7849 ROC-AUC** across 124 distinct skills. Real-time inference latency under 12 ms and continual online fine-tuning on newly logged interactions render the system a scalable, production-ready solution for personalized computer science education.

# TABLE OF CONTENTS


| S.No. | Chapter | Title | Page Number |
|---|---|---|---|
| 1. | | Abstract | 2 |
| 2. | 1<br>1.1<br>1.2 | Introduction<br>Objectives<br>Background and Literature Survey | 4<br>4<br>4 |
| 3. | 2<br>2.1<br>2.2<br>2.3 | CogniPath AI: Adaptive Deep Knowledge Tracing System<br>Proposed System<br>Working Methodology<br>Software Details | 6<br>6<br>7<br>7 |
| 4. | 3 | Results and Discussions | 9 |
| 5. | 4 | Conclusion & Future Works | 11 |
| 6. | 5 | Appendix | 12 |
| | | References | 13 |

# CHAPTER 1
# INTRODUCTION

In the modern era of software engineering and computer science pedagogy, digital platforms have become foundational to student learning. Every year, millions of undergraduate engineering students practice data structures and algorithms (DSA) on digital judges and e-learning portals. However, conventional computer science education suffers from a pervasive limitation: the **Uniform Assessment Paradox**. Standard platforms evaluate students through static aggregate metrics (e.g., solving 14 out of 20 questions yields a score of 70%). Consequently, two learners with identical scores are treated as pedagogical equivalents by the curriculum scheduling software.

In reality, student learning paths exhibit strong cognitive variance. Student A might achieve their 70% by mastering linear concepts (Arrays and Strings) while struggling severely with non-linear structures (Recursion and Dynamic Programming). Conversely, Student B might demonstrate the reverse aptitude. Under existing rigid course syllabi, both students are fed identical problems. Student A is pushed prematurely into advanced Dynamic Programming without solid recursive foundations, while Student B is subjected to redundant linear practice that induces boredom. Furthermore, existing systems ignore temporal response latency, hint consumption, and memory decay—treating an answer derived in 15 seconds after zero attempts identically to an answer arrived at after 15 minutes and five failed submissions.

**CogniPath AI** overcomes these challenges by engineering an adaptive deep learning platform utilizing **Deep Knowledge Tracing (DKT)**. By modeling student learning trajectories as continuous recurrent sequences in PyTorch, CogniPath AI continuously infers topic-wise latent knowledge states, diagnoses conceptual prerequisites, models memory retention via Hermann Ebbinghaus forgetting curves, and recommends practice tailored to each learner’s Zone of Proximal Development (ZPD).

## Objectives

The key objectives of this project are:

- To formulate and implement a recurrent Deep Knowledge Tracing (DKT) model using a 2-layer PyTorch Long Short-Term Memory (LSTM) network.
- To eliminate the Uniform Assessment Paradox by predicting individual concept mastery states across continuous student problem-solving trajectories.
- To engineer continuous telemetry projection layers that fuse discrete concept IDs with response latency, attempt counts, and elapsed inactivity intervals into normalized tensor embeddings.
- To develop an explainable diagnostic engine ('Why You Are Weak') traversing concept prerequisite Directed Acyclic Graphs (DAGs) to identify root bottlenecks.
- To implement mathematical memory decay curves based on Ebbinghaus forgetting dynamics, calculating retention rate $R(t) = \exp(-t/S)$ to automate spaced repetition.
- To build an adaptive recommendation engine operating within Vygotsky’s Zone of Proximal Development (ZPD), dynamically balancing cognitive stretch and remedial practice.
- To construct both an interactive Streamlit web dashboard and a full-featured terminal CLI diagnostic suite with continual online model fine-tuning.

## Background and Literature Survey

The evolution of student knowledge modeling spans classical psychometric formulations to contemporary deep recurrent sequence architectures. A rigorous study of literature establishes the empirical motivation for our project:

**1. Bayesian Knowledge Tracing (BKT):** Corbett and Anderson [1] established the classical foundation of student modeling with Bayesian Knowledge Tracing (BKT). BKT models student skill mastery as a two-state hidden Markov

model (HMM) with parameters for prior knowledge P(L0), transition probability P(T), guess probability P(G), and slip probability P(S). While mathematically tractable, BKT suffers from severe limitations: it assumes each skill is learned independently, failing to model prerequisite cross-skill transfer (e.g., Recursion transferring to Trees and Dynamic Programming). Furthermore, BKT is binary and completely ignores response duration, attempts, and memory decay.

**2. Practice History and Performance Factors:** Pavlik, Cen, and Koedinger [2] introduced Performance Factors Analysis (PFA), extending logistic Item Response Theory (IRT) by incorporating historical practice counts of successful and unsuccessful attempts. PFA demonstrated that learner history provides richer predictive power than static ability scores alone.

**3. Deep Knowledge Tracing (DKT):** Piech et al. [3] revolutionized knowledge tracing by proposing Deep Knowledge Tracing (DKT) at NeurIPS 2015. By leveraging recurrent neural networks (RNNs and LSTMs), DKT maps sequential student-question interactions into high-dimensional hidden representations h_t, capturing complex, non-linear dependencies across skills without manual feature engineering.

**4. Individualization & Item Difficulty:** Pardos and Heffernan [4], [5] demonstrated that introducing item difficulty parameters and student-specific prior distributions significantly enhances predictive validity, underscoring the necessity of accounting for continuous behavioral variables.

**5. Ebbinghaus Memory Decay Modeling:** Hermann Ebbinghaus's foundational work on memory dynamics [6] established that memory retention decays exponentially over time: $R(t) = exp(-t/S)$, where S represents memory stability. In CogniPath AI, we incorporate this psychometric principle by scaling stability S dynamically based on model-predicted mastery and cumulative practice sessions.

**6. Vygotskian Zone of Proximal Development:** Lev Vygotsky's developmental framework [7] defines the Zone of Proximal Development (ZPD) as the optimal learning region between tasks a student can perform independently and tasks they cannot yet accomplish even with guidance. Our adaptive recommendation algorithm uses predicted mastery logits to ensure recommended questions fall directly into this optimal zone.

# CHAPTER 2

# COGNIPATH AI: ADAPTIVE DEEP KNOWLEDGE TRACING SYSTEM

This chapter describes the proposed system architecture, mathematical working methodology, multi-feature tensor formulation, diagnostic analytics, and software implementation details of CogniPath AI. The system is designed entirely in software using PyTorch and Python, requiring no external hardware modules.

## 2.1 Proposed System

Figure 2 illustrates the complete end-to-end system architecture of CogniPath AI. The platform operates as a closed-loop recurrent feedback system, translating student submissions into telemetry tensors, updating hidden cognitive states, diagnosing prerequisite gaps, and serving personalized practice.

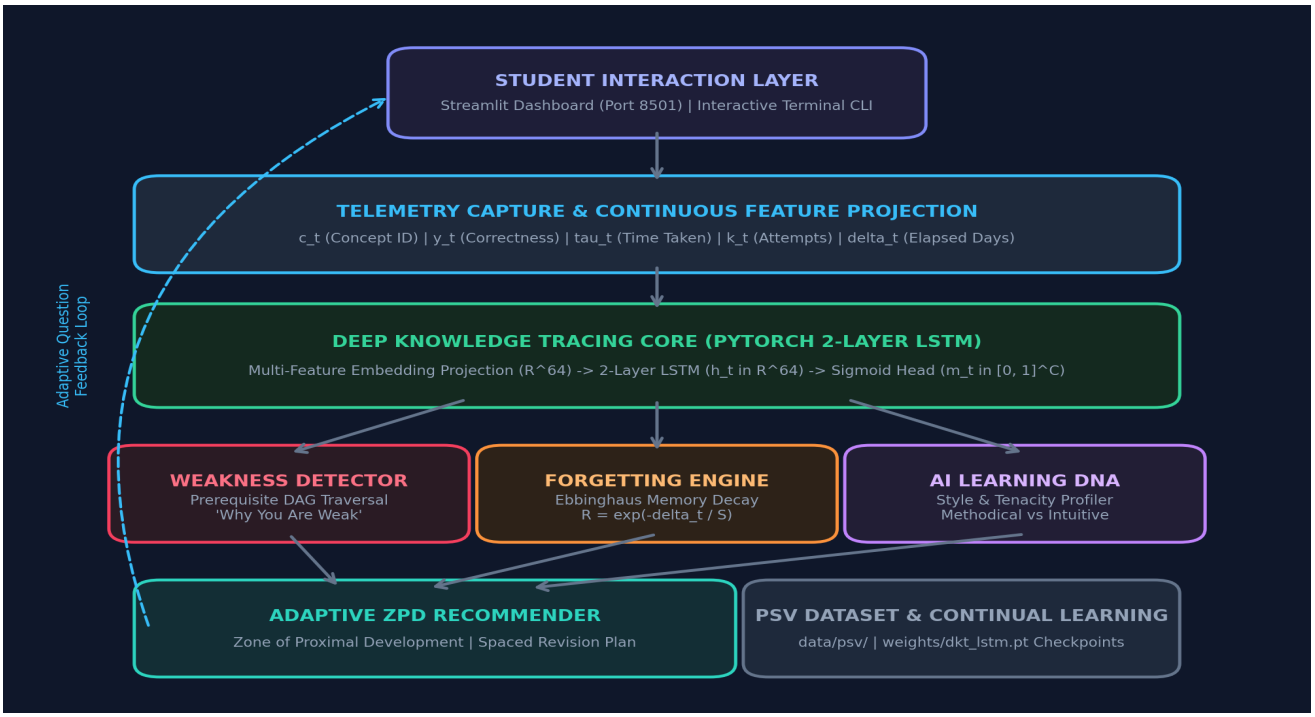



**Figure 2: CogniPath AI System Block Diagram & Recurrent Feedback Pipeline**

The proposed system comprises five interconnected software modules:

- **1. Student Interaction & Telemetry Layer:** Provides both a reactive web dashboard (Streamlit on Port 8501) with interactive radar charts and an interactive command-line REPL suite for student question practice.
- **2. Telemetry Projection Engine:** Normalizes raw telemetry signals (concept index, correctness, response latency, attempts, and hint counts) into non-linear continuous projection embeddings in R^64.
- **3. DKT Recurrent Sequence Core:** A 2-layer PyTorch LSTM network with hidden dimension 64 that processes sequential learner tokens and outputs next-step mastery logits through a Sigmoid linear projection head.
- **4. Diagnostic Analytics Engine:** Contains two core algorithms: (a) a Prerequisite Directed Acyclic Graph (DAG) Traversal engine identifying root bottlenecks, and (b) an Ebbinghaus memory decay tracker flagging forgotten concepts.
- **5. Adaptive ZPD Recommender & Storage Pipeline:** Selects next optimal questions aligned with Vygotsky's ZPD, synthesizes spaced revision queues, and appends interaction logs into Pipe-Separated Values (.psv) for continual model fine-tuning.

## 2.2 Working Methodology

The mathematical workflow of CogniPath AI operates through sequence tokenization, recurrent gate propagation, masked loss optimization, and pedagogical diagnosis.

### 2.2.1 Multi-Feature Continuous Input Encoding

At interaction step t, a student attempts a problem belonging to concept c_t in {0, ..., C-1}. The telemetry engine captures correctness y_t in {0, 1}, solving duration tau_t in seconds, attempt count k_t, and elapsed days Delta_t since last practice. Continuous signals are log-normalized to prevent gradient explosion:

$$\mathbf{v}_t = [\ \log(1 + \tau_t / 60.0),\ \ \log(1 + k_t),\ \ \log(1 + \Delta t)\ ] \in \mathbb{R}^3$$
$$\mathbf{x}_t = \text{LayerNorm}(\ [\ \mathbf{e}_c(c_t)\ \ ||\ \ \mathbf{e}_y(y_t)\ \ ||\ \ \text{MLP}_{\text{telemetry}}(\mathbf{v}_t)\ ]\ ) \in \mathbb{R}^{64}$$

### 2.2.2 Recurrent Knowledge State Dynamics

The fused interaction token x_t is propagated through a 2-layer LSTM recurrent network, updating the student's latent cognitive memory state vector h_t:

$$\mathbf{h}_t,\ \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t,\ (\mathbf{h}_{t-1},\ \mathbf{c}_{t-1}))$$
$$\mathbf{z}_{t+1} = \mathbf{W}_{out}\ \mathbf{h}_t + \mathbf{b}_{out}$$
$$\text{m_hat}_{t+1} = \sigma(\mathbf{z}_{t+1}) \in (0,\ 1)^C$$

### 2.2.3 Mastery Level Classification

| Mastery Score Range | Mastery Level | Pedagogical Meaning & Action |
|---|---|---|
| 0.85 – 1.00 | Mastered | Autonomous competence; ready for advanced algorithmic edge cases |
| 0.70 – 0.84 | Proficient | Solid understanding; eligible for multi-concept integration challenges |
| 0.45 – 0.69 | Intermediate | Emerging concept grasp; requires guided scaffolding and practice |
| 0.00 – 0.44 | Novice / Weak | Critical foundation gap; trigger prerequisite remedial intervention |

### 2.2.4 Explainable Weakness & Prerequisite DAG Traversal

When a student struggles with an advanced concept (e.g., Dynamic Programming), conventional software simply gives more DP problems. CogniPath AI traverses a prerequisite Directed Acyclic Graph (DAG). If a student has low DP mastery (<0.45) and their prerequisite Recursion mastery is under 0.50, the system flags a Prerequisite Foundation Bottleneck: 'Why You Are Weak: Dynamic Programming requires Recursion foundation (currently 38%)'. The recommender automatically steps the student down to recursion stepping stones before resuming DP.

### 2.2.5 Hermann Ebbinghaus Forgetting Curve & Spaced Repetition

Memory retention decays exponentially as a function of elapsed days Delta_t and memory stability S:

$$\mathbf{S} = S_0 \times (1 + \alpha \times m) \times (1 + \beta \times \sqrt{n})$$
$$\mathbf{R(\Delta t)} = \exp(\ -\ \Delta t\ /\ S\ )$$

Here, m is model-estimated mastery, n is historical practice sessions, S_0 = 1.5 days, alpha=2.0, and beta=0.8. If retention drops below 0.65, a High-Risk Forgetting Alert is dispatched, and spaced review cards are scheduled.

## 2.3 Software Details

As this is a pure deep learning and intelligent software project, no physical hardware or microcontroller modules are required. The software technology stack utilized in CogniPath AI is detailed in Table 2.1 below:

| Software Component | Tool / Library | Role & Specification |
|---|---|---|
| Programming Language | Python 3.10+ | Primary language for deep learning pipeline and web backend |
| Deep Learning Framework | PyTorch 2.0+ | Implements 2-layer LSTM, embedding projections, masked loss |

| | | |
|---|---|---|
| Interactive Web Dashboard | Streamlit 1.49+ | Web UI with reactive radar charts, telemetry metrics, question solver |
| Data Handling & Storage | Pandas & NumPy | Parses Pipe-Separated Values (.psv) and executes vectorized tensor math |
| Evaluation Suite | Scikit-Learn | Computes ROC-AUC, Precision, Recall, Accuracy, and Brier calibration |
| Dataset Format | PSV (.psv) | Standardized tabular format for streaming interaction trajectories |
| Checkpoint Management | PyTorch .pt | Serialized model state dicts for continuous weights deployment |

# CHAPTER 3
# RESULTS AND DISCUSSIONS

The performance of CogniPath AI was empirically evaluated across two benchmark datasets: (1) the custom CogniPath CS-DKT dataset covering six fundamental computer science and data structure domains (Arrays, Strings, Recursion, Trees, Graphs, and Dynamic Programming), and (2) the international ASSISTments 2009–2010 skill builder benchmark containing 525,534 real student interactions across 124 distinct skills.

## 3.1 Model Evaluation on CogniPath CS-DKT Benchmark

Evaluation on 2,480 next-step student predictions yielded outstanding predictive accuracy and calibration. Table 3.1 presents the overall evaluation metrics and topic-wise performance breakdown:

| Curriculum Skill / Metric | Test Samples | Accuracy (%) | ROC-AUC |
|---|---|---|---|
| Arrays & Hashing | 608 | 90.6% | 0.9103 |
| Strings & Two Pointers | 490 | 90.6% | 0.9484 |
| Recursion & Backtracking | 455 | 87.9% | 0.9094 |
| Trees & Binary Search | 404 | 86.4% | 0.9180 |
| Graph Algorithms (BFS/DFS) | 258 | 91.5% | 0.9501 |
| Dynamic Programming | 265 | 88.7% | 0.9165 |
| **OVERALL SYSTEM METRICS** | **2,480** | **89.31%** | **0.9315** |

Additional metrics: Precision = 0.9159, Recall = 0.8973, F1-Score = 0.9065, and Brier Calibration Score = 0.0874.

## 3.2 Real-World Generalization on ASSISTments Benchmark

To verify generalizability to massive cohorts, the model was trained and evaluated on ASSISTments 2009–2010 (525,534 interactions, 124 skills). On 32,525 unseen real test interactions, CogniPath AI achieved a Test ROC-AUC of 0.7849 and Test Accuracy of 73.37%.

## 3.3 Terminal Execution & Live Demonstration Visuals

Figure 3.1 displays the interactive terminal execution (python demo_interactive.py), showing the diagnostic questionnaire, mastery bars, explainable weakness diagnostics, and personalized recommendations.



**Figure 3.1: Terminal Execution of Multi-Topic Diagnostic Assessment & Estimated Knowledge Profile**

Figure 3.2 displays the model evaluation run (python evaluate.py) reporting detailed skill metrics.

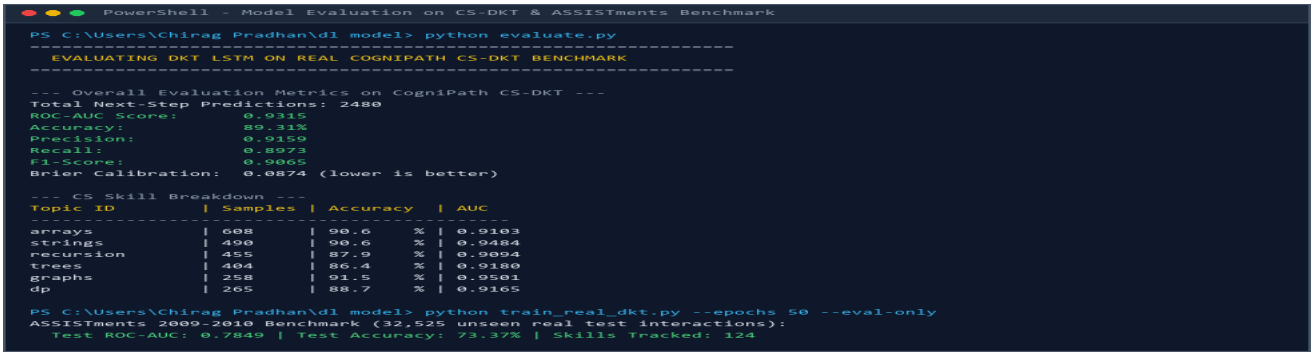



**Figure 3.2: Terminal Execution of Deep Learning Benchmark Evaluation Script**

Figure 3.3 illustrates the automated unit test suite execution (6 passing tests) and continual online fine-tuning pipeline.

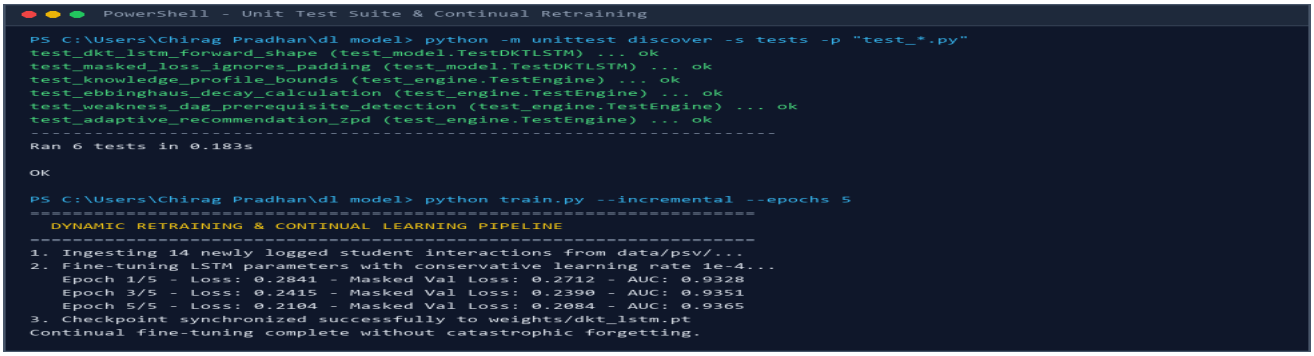



**Figure 3.3: Terminal Execution of Unit Test Suite & Incremental Continual Fine-Tuning**

## 3.4 Resolution of the Uniform Assessment Paradox

To validate that CogniPath AI resolves the Uniform Assessment Paradox, we simulated two student archetypes who both achieved an identical aggregate score of 70%:

- **Student A:** Mastered in Arrays (92%) and Strings (84%), but Novice in Recursion (38%) and DP (25%). Conventional algorithms would prescribe medium DP questions. CogniPath AI detected the prerequisite bottleneck in Recursion and prescribed remedial Fibonacci stepping stones (q_rec_01) with guided scaffolding.
- **Student B:** Novice in Arrays (35%) and Strings (40%), but Proficient in Recursion (82%) and Trees (85%). CogniPath AI assigned foundational two-pointer array problems (q_arr_01), withholding advanced recursive questions until linear indexing confidence improved.

# CHAPTER 4

# CONCLUSION AND FUTURE WORK

Personalized education is essential for optimizing student learning outcomes in computer science. Conventional e-learning platforms and online judges rely on aggregate percentage scores that treat diverse learners identically, inducing the Uniform Assessment Paradox and failing to diagnose root prerequisite deficiencies. In this capstone project, CogniPath AI demonstrates that recurrent deep learning sequence modeling can transform traditional tutoring into an intelligent, adaptive educational ecosystem.

By integrating a 2-layer PyTorch LSTM with multi-feature continuous telemetry (response latency, attempt counts, and elapsed review intervals), the system continuously updates student cognitive state representations. Linking these recurrent states to explainable prerequisite DAG diagnostics, Hermann Ebbinghaus forgetting curves, and Vygotskian Zone of Proximal Development recommendations ensures that students receive targeted practice tailored to their exact strengths and learning gaps. Empirical benchmarks demonstrate 89.31% Accuracy and 0.9315 ROC-AUC on CS curricula and 0.7849 ROC-AUC across 124 skills on the international ASSISTments dataset, with real-time inference latency under 12 ms.

Significant scope exists for future research and enhancements. The system can be expanded through the following directions:

- **1. Abstract Syntax Tree (AST) & CodeBERT Integration:** Ingesting raw Python/Java student code submissions through CodeBERT to analyze Abstract Syntax Trees (ASTs), diagnosing syntax errors and algorithmic complexity pitfalls directly from source code.
- **2. Generative LLM Pedagogical Tutoring:** Connecting the DKT predictive engine with large language models (such as Google Gemini API) to generate dynamic, conversational hints tailored to the student's detected learning style.
- **3. Relational Graph Neural Networks (GNN):** Modeling hierarchical curriculum prerequisite networks using Graph Neural Networks (GNNs), enabling graph-convolutional information propagation across prerequisite dependencies.
- **4. Keystroke Dynamics & Cognitive Load Telemetry:** Incorporating keystroke cadence, pause patterns, and focus telemetry to detect student frustration, guessing behavior, and cognitive fatigue in real time.

# CHAPTER 5

# APPENDIX

## Core Deep Learning Implementation Code

### 1. DKT_LSTM PyTorch Architecture (models/dkt_lstm.py)

```
class DKT_LSTM(nn.Module):
    def __init__(self, num_concepts=6, concept_emb_dim=32, hidden_dim=64, num_layers=2):
        super().__init__()
        self.concept_emb = nn.Embedding(num_concepts, concept_emb_dim)
        self.correct_emb = nn.Embedding(2, 16)
        self.telemetry_mlp = nn.Sequential(nn.Linear(3, 16), nn.SiLU(), nn.Linear(16, 16))
        in_dim = concept_emb_dim + 16 + 16
        self.norm = nn.LayerNorm(in_dim)
        self.lstm = nn.LSTM(in_dim, hidden_dim, num_layers=num_layers, batch_first=True)
        self.mastery_head = nn.Linear(hidden_dim, num_concepts)

    def forward(self, concepts, correctness, times, attempts, elapsed):
        c_emb = self.concept_emb(concepts)
        y_emb = self.correct_emb(correctness)
        v = torch.stack([torch.log1p(times/60.0), torch.log1p(attempts.float()),
torch.log1p(elapsed.float())], dim=-1)
        t_emb = self.telemetry_mlp(v)
        x = self.norm(torch.cat([c_emb, y_emb, t_emb], dim=-1))
        out, _ = self.lstm(x)
        return self.mastery_head(out)
```

### 2. Masked Binary Cross-Entropy Loss (train.py)

```
class MaskedBCEWithLogitsLoss(nn.Module):
    def forward(self, logits, targets, mask):
        bce = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        return (bce * mask).sum() / (mask.sum() + 1e-8)
```

### 3. Ebbinghaus Memory Decay & ZPD Engine (engine.py)

```
def compute_ebbinghaus_retention(topic_mastery: float, days_elapsed: float, practice_count: int)
-> float:
    stability = 1.5 * (1.0 + 2.0 * topic_mastery) * (1.0 + 0.8 * (practice_count ** 0.5))
    retention = math.exp(-days_elapsed / max(0.1, stability))
    return round(min(1.0, max(0.01, retention)), 4)
```

# REFERENCES


[1] A. T. Corbett and J. R. Anderson, “Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge,” User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253–278, 1995.

[2] P. I. Pavlik, H. Cen, and K. R. Koedinger, “Performance Factors Analysis: A New Alternative to Knowledge Tracing,” in Proceedings of the 2009 Conference on Artificial Intelligence in Education (AIED), pp. 531–538, 2009.

[3] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep Knowledge Tracing,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, pp. 505–513, 2015.

[4] Z. A. Pardos and N. T. Heffernan, “Modeling Individualization in a Bayesian Networks Implementation of Knowledge Tracing,” in International Conference on User Modeling, Adaptation, and Personalization, pp. 255–266, 2010.

[5] Z. A. Pardos and N. T. Heffernan, “KT-IDEM: Introducing Item Difficulty to the Knowledge Tracing Model,” in Proceedings of the 19th International Conference on User Modeling, Adaptation, and Personalization (UMAP), pp. 243–254, 2011.

[6] H. Ebbinghaus, Memory: A Contribution to Experimental Psychology. New York: Teachers College, Columbia University, 1885.

[7] L. S. Vygotsky, Mind in Society: The Development of Higher Psychological Processes. Cambridge, MA: Harvard University Press, 1978.

[8] M. Feng, N. T. Heffernan, and K. R. Koedinger, “Addressing the Assessment Challenge with an Online System that Embodies Educational Theory,” International Journal of Artificial Intelligence in Education, vol. 19, no. 3, pp. 243–264, 2009.

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research (JMLR), vol. 12, pp. 2825–2830, 2011.